Day 10 – React Detailed Notes

Complete theory reference for React fundamentals and the React To-Do List practical.

1. What is React?

Definition: React is a JavaScript library used to build user interfaces, especially interactive web applications.

Mental model: Data/State → React → UI. When state changes, React updates the UI.

Important: React does not replace HTML, CSS and JavaScript. It uses JavaScript to manage the UI and JSX for HTML-like syntax.

2. Why React is Used

Reusable components: Build UI using reusable pieces such as Navbar, Button, TodoItem and Card.

State-based UI: Update state instead of manually changing many DOM elements.

Maintainability: Large applications can be divided into smaller components.

Dynamic interfaces: Useful when the screen changes based on user actions or data.

3. React vs Traditional JavaScript

Traditional JavaScript: Select and manually modify DOM elements using methods such as getElementById(), createElement(), appendChild() and removeChild().

React: Describe what the UI should look like for the current state. When state changes, React re-renders the component and updates the UI.

Example: Traditional JS manually creates/removes list elements. React stores tasks in state and maps them to UI.

4. Setting Up React with Vite

Vite: A modern development/build tool used to create and run React projects quickly.

Typical commands:

```
npm create vite@latest react-todo -- --template react
cd react-todo
npm install
npm run dev
```

5. React Project Structure

package.json: Project information, dependencies and scripts.

node_modules: Installed packages generated by npm.

public: Public/static files.

src: Main source-code folder.

main.jsx: Entry point that starts the React application.

App.jsx: Main/root component.

index.css: Global styling.

components/: A common folder for reusable components.

6. JSX

Definition: JSX is HTML-like syntax written inside JavaScript.

Expressions: JavaScript expressions go inside { }.

Differences: Use className instead of class and htmlFor instead of for.

Return: A component returns one overall JSX structure, using a parent element or fragment when needed.

7. Components

Definition: A component is a reusable UI building block.

Functional component: A JavaScript function that returns JSX.

Naming: Component names normally start with a capital letter.

Rendering: Use JSX such as .

Composition: Components can be placed inside other components.

8. Props

Definition: Props are values passed from a parent component to a child.

Direction: Parent → Child.

Read props: Props can be received through the component parameter or destructuring.

Read-only: A child should not directly modify the props it receives.

```
function Welcome({ name }) {
  return <h1>Hello {name}</h1>;
}

function App() {
  return <Welcome name="Namratha" />;
}
```

9. Event Handling

Purpose: Events respond to actions such as clicking and typing.

Common events: onClick, onChange and onSubmit.

Handler: Use onClick={handleClick}.

Important: Avoid onClick={handleClick()} for a normal event handler because it calls the function immediately.

```
function App() {
  function handleClick() {
    console.log("Button clicked");
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

10. useState()

Definition: useState is a React Hook for storing data that can change.

Syntax: const [state, setState] = useState(initialValue);

Values: The first value is current state; the second is the updater function.

Re-render: Calling the updater tells React that the UI may need to be rendered again.

To-do example: const [tasks, setTasks] = useState([]);

```
import { useState } from "react";

const [count, setCount] = useState(0);
```

11. Updating State

Correct: Use the setter function rather than directly changing the state variable.

Example: setCount(count + 1). For arrays, create a new array when adding/removing items.

Reason: React uses state updates to know when the UI should update.

12. Conditional Rendering

Meaning: Show different UI depending on a condition.

Ternary: condition ? :

Logical AND: condition &&

To-do: If tasks.length is 0, show “No tasks available”; otherwise show the list.

```
{tasks.length === 0
  ? <p>No tasks available</p>
  : <TodoList tasks={tasks} />}
```

13. Lists with map()

Purpose: map() creates UI elements from an array of data.

Example: tasks.map(task =>).

Result: When the array changes, React renders the updated collection.

```
tasks.map(task => (
  <TodoItem key={task.id} task={task} />
))
```

14. key in Lists

Purpose: A key gives each list item a stable identity so React can track items efficiently.

Best choice: Use a unique, stable value such as task.id.

Avoid: Using changing values as keys; array indexes can cause problems when items are inserted, removed or reordered.

15. Parent → Child Data Flow

Structure: App → TodoForm / TodoList → TodoItem.

State ownership: App can own the tasks state.

Data: App passes tasks or task data through props.

Callbacks: App can pass functions such as onAdd, onDelete and onComplete to children.

16. To-Do List Component Structure

App.jsx: Owns tasks state and coordinates the application.

TodoForm.jsx: Input and Add button; handles user input.

TodoList.jsx: Receives tasks and maps over them.

TodoItem.jsx: Represents one task and provides complete/delete actions.

index.css: Basic styling.

17. To-Do List State Design

Task object: { id, text, completed }.

Add: Create a new task and update the tasks array.

Delete: Create a new array excluding the selected task.

Complete: Create a new array with the selected task's completed value changed.

```
const newTask = {
  id: Date.now(),
  text: taskText,
  completed: false
};
```

18. Complete React Data Flow

1. User types a task in TodoForm.

2. User clicks Add.

3. TodoForm calls a callback received through props.

4. App updates tasks using setTasks().

5. React re-renders.

6. TodoList receives updated tasks.

7. map() creates a TodoItem for each task.

8. TodoItem calls parent callbacks for delete/complete.

19. Common Beginner Mistakes

Use PascalCase component names: TodoItem, TodoList.

Use onClick={handleClick}, not onClick={handleClick()} for a normal handler.

Do not directly mutate state arrays with tasks.push(...); use the state setter.

Give mapped elements a stable unique key.

Use className instead of class in JSX.

Check import/export names and component file paths.

20. Day 10 Practical Checklist

Create the Vite React project.

Create TodoForm, TodoList and TodoItem components.

Use useState for tasks.

Add, delete and complete tasks.

Use props for data and callback functions.

Use button/input events.

Use map() with a unique key.

Show “No tasks available” when the list is empty.

Add basic clean CSS.

21. Quick Revision

React: JavaScript library for building UIs.

JSX: HTML-like syntax inside JavaScript.

Component: Reusable UI building block.

Props: Parent → child data.

State: Data that changes and affects UI.

useState: Hook for creating/updating state.

Events: Respond to user actions.

map: Create UI from an array.

key: Stable identity for list items.

Conditional rendering: Show UI based on a condition.